

COMPENG 3DY4 FINAL REPORT

GROUP 05 | APRIL 2, 2026

Logan Eby-400472673 | James Cameron-400450866 Justin Shin-400467486 | Matthew Rakic-400449728

Introduction: The COMPENG 3DY4 project focuses on the design and implementation of a software-defined radio (SDR) frequency-modulated (FM) receiver that operates using RTL-SDR hardware and a Raspberry Pi 4 system. The system is designed to receive FM signals and produce mono audio, stereo audio, and Radio Data System (RDS) output, depending on the selected mode and processing path. Each group must support a set of assigned operating modes with specific sampling and processing constraints, which need to be usable based on the input settings when running the system. This requires using concepts developed in earlier labs to produce an implementation that can continuously process incoming samples at a rate fast enough to keep up with live radio reception. The project requires working with complex specifications while implementing a real-time computing system.

Overview: FM broadcasting carries multiple types of information within a single radio channel. In addition to the main mono audio content occupying the baseband region from 0 to 15 kHz, an FM broadcast can also contain a 19 kHz pilot tone for stereo synchronization, a stereo subchannel centred around 38 kHz, and an RDS subchannel centred around 57 kHz for low-rate digital information such as station metadata and text. As a result, an FM receiver must selectively isolate and process different parts of the demodulated spectrum depending on the desired output.

In this project, these tasks are implemented using a software-defined radio (SDR) approach, with signal processing performed in software. The RTL-SDR front end acquires the radio-frequency signal and provides interleaved in-phase and quadrature samples to the Raspberry Pi, where the receiver is constructed as a chain of digital signal processing blocks. At a high level, the radio-frequency (RF) front end first extracts the desired FM channel and demodulates it. The demodulated signal is then routed into separate processing paths for mono audio, stereo audio, and RDS, depending on the selected mode of operation and enabled receiver path.

The implementation of these paths relies on several digital signal processing (DSP) building blocks. Finite impulse response (FIR) filters are used to isolate specific frequency regions and suppress unwanted spectral content, while decimation and rational resampling are used to convert signals between sample rates appropriate for later stages of processing. In the stereo and RDS paths, carrier recovery is required before down conversion, which is achieved using phase-locked loops (PLLs) and numerically controlled oscillators (NCOs). Once the relevant subchannels are shifted to baseband and filtered, the mono and stereo paths produce audio output, while the RDS path continues through symbol recovery, bit decoding, and frame synchronization to extract transmitted digital data.

Implementation: The labs developed several functions which would serve as basic building blocks for the project. We developed the function used to compute the impulse response "h" for the low-pass filter, based on the sinc function. The rectangular low-pass cut-off band is approximated based on the sampling frequency, the cut-off frequency, and the number of taps. This function was a crucial building block, as it was used to generate the impulse response for all low-pass filters across the entire project modelling and implementation. Along with this, we implemented several versions of digital filtering using convolutions, including single-pass and block processed, with various methods and degrees of optimization. The labs also explored FM channel extraction functions for the RF front-end, establishing a crucial building block whose output feeds all three computing paths: Mono, Stereo, and RDS. fmDemodArctan was also developed as an accurate demodulator. Given this, the cleaner signal extraction it provides comes at no meaningful expense, improving output quality across all three processing paths. Lab 3 built the Python model for Mono path. The Python model was a helpful reference throughout the project, and unit testing/benchmarking building blocks were crucial to ensure that the functions were correctly implemented and meeting performance standards.

Mono Implementation: The first step we took was to set the file up for Unix command-line operation using `argc` and `argv`, allowing the mode and signal format to be passed in at runtime rather than

hardcoded. From the parsed mode, we derived all downstream parameters using a switch-case statement, for sampling frequency, up/down sampling factors, block size, and filter cutoff frequencies, so that the entire processing path could be set from a single command-line argument. In the first iteration, filter coefficients were generated using the **impulseLPF** function, and digital filtering was performed using the unoptimized block convolution developed in the labs. The resampling was handled in discrete, sequential steps rather than as a unified operation. Upsampling was performed by a zero-insertion loop, followed by filtering, then a separate decimation loop to downsample the filtered output. This approach was a baseline before optimization. In the second iteration, the zero-insertion upsampling step was retained, but filtering and downsampling were consolidated into the first version of the **optimizedFilt** function. By taking the decimation factor as an additional parameter, the function computes only the indices that survive downsampling, reducing the total number of operations compared to the first iteration. In the third iteration, the zero-padded upsampling vector was eliminated entirely through the introduction of the **polyResampler** function. Filtering a zero-padded vector wastes significant computation time, as most operations evaluate to zero and contribute nothing to the output. This function is further explained in the Optimizations and Multithreading section. In the final Mono implementation, ready for real-time application, the **optimizedFilt** function was retained for Modes 0 and 1, as their upsample factor of 1 requires no upsampling. For Modes 2 and 3, the **polyResampler** function was used instead. With this optimization in place, the program successfully executed in real-time on the R-PI 4. The main issue with this step was that Mode 1 had a faint yet noticeable high-pitched tone ringing in the output audio. This tone would be more noticeable at low numTaps and eliminated at numTaps outside of the allowed range. This was a mix of the unique samples we were using, along with our unique intermediate and audio frequencies. When the intermediate frequency is so high in comparison to the audio frequency, the percentage of the cutoff window gets lower, and the low-pass filter has less time to fall off. As a result, the stereo pilot tone is included in the filter. This then leads it to alias into the mono output at 13kHz, as when our audio frequency is 32kHz, our Nyquist frequency is 16 kHz, lower than the 19kHz that our pilot tone is at. With this, if the mono audio already has a high frequency at that tone, the pilot tone gets added to it, causing a slight, consistent high pitch. This couldn't be rectified within the project constraints and was allowed by Dr. Nicolici.

Stereo: When modelling stereo, we skipped a single-pass implementation and went straight to a block implementation, which complicated debugging. The model's overall design was straightforward to implement, as fmPLL was already mostly designed for us, and band-pass filters required only a small change to the firwin coefficients. With this implemented, we then turned to fixing the PLL implementation to work with blocking. We tried various setups, and what proved most reliable before full troubleshooting was passing a states array to save all variables on each pass. The model did not work with everything having state-saving implemented. Through code review, we found that **bufferedFmState** was too large. This size mismatch with stereo led to the combination of mono and stereo not behaving as expected. Once this was rectified by initializing **bufferedFmState** as $\text{int}((\text{audio_taps} - 1) / 2)$, the model worked as expected. With the model fully validated and working, the implementation was quite straightforward, mostly just involving refactoring the Python code to C++. At this stage, the **optimizedFilt** function was further optimized, as detailed in the Optimizations and Multithreading section, and the stereo path was integrated with the Mono path to form the complete Audio Path, supporting all modes and formats configurable via command line. When implementing Stereo, one of the last issues we ran into was that, unlike the model, the left and right channels were slightly bleeding into one another. Upon debugging, we found that our carrier and channel mixing were desynchronized due to an incorrect indexing input. Specifically, we were doing `mixedStereoFilt.push_back(2 * scarPilotI[i + 1] * schFilt[i])`. Once this was fixed by removing the +1, the left and right channels were distinct with no bleeding, and stereo was working. With this, the program achieved real-time execution on the Raspberry Pi 4.

RDS: The main challenges at this stage stemmed from the CDR block and the data processing steps, debugged primarily by examining the post-RRC waveform, constellation diagram, and RDS text output. Initial validation was performed using samples 3 and 8, which had stronger RDS signals, allowing the later stages to be developed before revisiting the CDR and decoding schemes for weaker signals. In the first iteration, the CDR was implemented without timing error detection, sampling at fixed intervals of SPS. The initial sample position was set manually by plotting the post-root-raised cosine (RRC) signal and identifying the approximate location of the first peak. This was considered acceptable at this stage, as the samples were assumed to be free of timing error. However, the constellation graph was quite dispersed, meaning that the algorithm needed development. The Manchester decoding scheme processes symbols from the CDR output, each either 1 or -1, into bits by examining consecutive pairs, then advancing by 2 after each. In the first iteration, the same-polarity symbol pair case was naively handled by shifting the pairing window by a single symbol to recover alignment. However, it was later understood that this approach inevitably discards information and is therefore not suitable. The differential decoding block, which follows Manchester decoding, operates by XORing each bit in the output bitstream with its predecessor, recovering the original data from the differentially encoded signal. The first improvement in the second iteration was the introduction of state-saving between blocks to preserve the last symbol and bit from each processed block. In Manchester decoding, the algorithm pairs each symbol with its successor, meaning that if the index lands on the final symbol of a block, that symbol must be paired with the first symbol of the next block rather than discarded. State-saving was implemented to carry this boundary symbol forward when the index reaches the end of the array. Similarly, since differential decoding XORs each bit with its predecessor, the final bit of each block is saved as a state to ensure continuity into the next. This iteration also introduced two key improvements to the CDR: a Timing Error Detection mechanism and a more robust method for determining the initial sampling position. For timing error detection, the Mueller-Müller algorithm was adopted following research outside of course material¹. The algorithm operates by sampling the RRC waveform at intervals determined by a running offset, adjusting the sampling phase based on detected timing error at each symbol transition. Specifically, when a symbol change is detected between consecutive samples, the timing error is computed as:

$$error = symbol_{prev} \cdot sample_{curr} - symbol_{curr} \cdot sample_{prev}$$
 If no symbol change occurs, no phase information can be extracted, and the timing error is set to zero¹. The computed error is scaled by a proportional gain K and accumulated into a running offset, which is clipped to $\pm SPS/2$ to prevent excessive deviation. This offset is then added to the step size of SPS to shift the next sampling point, allowing the algorithm to continuously correct for timing drift. At this stage, however, the offset was not saved across blocks. The initial sampling position was determined by selecting the index of the absolute maximum from a window of SPS samples. To compensate for the absence of state-saving across blocks, this initial position was recomputed at the start of every block. However, this did not lead to effective clock and data recovery, leading to incorrect down stream data.

In the third iteration, state-saving was introduced for the CDR offset. The resulting constellation diagram demonstrated a near-ideal output, with significantly tighter clustering around the expected symbol points compared to the previous iteration. At this stage, the TAs advised incorporating a settling period of approximately 0.5 seconds (roughly 13 blocks for Mode 0) before beginning CDR and data processing. This allows the PLL to lock and the signal to stabilize, ensuring the highest quality input data from the outset. While the decoding scheme had not yet been finalized, RDS Frame Synchronization was developed in parallel as a class. Although this required some review of Python classes and object-oriented

¹ However, this was a mistake, as highlighted during the cross-examination; phase information can be extracted for no-symbol-change pairs

programming, the implementation was relatively straightforward, detecting sync offsets via a sliding window and tracking synchronization state through a Finite State Machine (FSM).

The final iteration added an improvement to the Manchester decoding scheme to replace the naive phase correction described earlier. Rather than simply shifting the pairing window by one symbol for same-polarity pairs, the scheme was redesigned around two explicit states: **locked** and **unlocked**. When **unlocked**, the correct pairing phase has not yet been established. The decoder therefore evaluates both possible phases over a window of 24 symbols, counting the number of same-polarity pairs in each. The phase yielding fewer same-polarity pairs is selected as the correct one, its corresponding bits are passed to the differential decoder, and the state transitions to **locked**. Once **locked**, the decoder proceeds with the chosen phase, but to guard against loss of synchronization during decoding, the number of consecutive same-polarity pairs is monitored. If two such pairs are detected consecutively, all symbols from the symbol immediately preceding the first of the two consecutive same-polarity pairs through to the end of the current block are saved, and the decoder breaks out of the current processing loop with the state transitioning back to **unlocked**. On the next iteration, the symbols extracted from the next post-RRC waveform are obtained, and the saved symbols are prepended to them before decoding begins, allowing the decoder to re-establish the correct phase without discarding any information. With the working CDR algorithm, corrected decoding scheme, and the frame synchronization and application layer built, the model was able to extract valid RDS data from the samples 2, 3, and 8. However, the RDS data extraction from other samples were not as successful. This was attributed to the fact that these had weaker RDS signals, therefore the data was harder to accurately extract.

Learning from the mistake of going straight to block modelling in stereo, modelling first started in single pass. A class was created for the RDS application, alongside a table for the PTY codes and a bit-to-integer conversion helper function. This class takes a block window and a block offset from valid blocks and decodes them to display the correct information. This is done by pushing blocks onto a group queue and extracting the bits corresponding to our desired info from it. This info is then converted to the correct type for display. For Pi code, the bits from block A get converted to hexadecimal on output. For the PTY name, bits 6-10 from block B are used to navigate a table of PTY names. For the PS name, we check whether the group type and version are 0 using bits 0 to 4 from block B, and slot the characters from block D, bits 0 to 15, into their corresponding segment of the output string using the segment marker from block B, bits 14 and 15. Debugging was difficult to do while the state of the rest of the RDS model was uncertain, but once it was working, we found that RDS application ran without any errors. With this, we brought it over to blocking. This was a simple transfer, as the RDS application already operated on a per-block basis, with blocks being smaller blocks output by the process block. As a result, no state saving was needed, and it worked as it had in single-pass mode. Instead of implementing everything in one block, as the model was done, we decided to break the model into smaller helper functions that matched the major stages of the signal processing chain. These functions were defined in `rds_helpers.cpp` and handled carrier recovery, demodulation, clock and data recovery, as well as Manchester and differential decoding. This approach made the implementation much easier to manage, since each stage could be developed and debugged independently without affecting the rest of the pipeline. Due to the higher complexity of frame synchronization and the application layer, these components were implemented separately using their own classes. The `rds_thread.cpp` file was then used to coordinate all functions and classes, and with this structure, the thread file was clean and easy to follow due to all the logic being done within the helpers and classes. Finally, all filter coefficients and variables were initialized in `project.cpp`, where the necessary parameters were passed into the RDS thread. The thread was then run in parallel with the audio thread to enable real-time execution through multithreading. While performing refactoring on the main project file, to maintain organization all filter coefficients were grouped within a struct. These were then initialized

using a function in filter.cpp. Importing RDS Application into the integration proved to be more difficult than the Python model programming. This is because we relied on a few Python-native functions that were not available in C++. As a result, we needed to add a few helper functions and test them to ensure that we could more simply import RDS Application. These include a function that converts an integer input to hexadecimal for display in PTY, as well as a function that serves as the slice function in Python, allowing us to extract specific chunks of bits. To improve symbol decoding accuracy for weaker signals, the Manchester decoding scheme was updated during the implementation stage. Since the previous version was hardcoded to separate symbols solely based on its sign, it fails for weaker signals². To address this, the updated implementation performs a relative comparison within each symbol pair. The difference between the two symbols is computed and normalized by the sum of the pair's absolute values. This normalization ensures that the resulting value always falls within a consistent range regardless of the overall signal strength, making the configurable threshold applicable across strong and weak signals. The threshold value of 0.25 was determined empirically based on the RDS text output quality observed across multiple samples, producing the most accurate decoding results across varying signal strengths. In both the model and the design, it was decided to pass the raw sample values from the CDR stage directly into the Manchester decoder, rather than converting them to hard symbols of 1 or -1 at the CDR stage. This preserves the relative amplitude information between consecutive samples, which is what the normalized difference relies upon. The next issue was that the output degraded entirely when the number of filter taps was reduced to 75. To investigate, the post-RRC signal was plotted in gnuplot, which revealed significant high frequency noise. This was initially perplexing, as the Python model operated correctly at the same tap count. The root cause was traced back to the PLL function, where the normalized bandwidth had been hardcoded to 0.01, which was appropriate for audio but far too wide for RDS. The RDS path requires a much tighter loop bandwidth of 0.001 to operate correctly. With the normalized bandwidth being too wide, reducing the number of taps widened the transition band of the filter, allowing high frequency components to pass. The fix was straightforward: the normalized bandwidth is used as a parameter rather than being hardcoded, allowing it to be set appropriately and independently for the audio and RDS paths.

Optimizations and Multithreading

Polyphase Resampling: Unlike standard C++ optimizations such as loop unrolling and precomputed ranges, the **polyphase filter** originates from a conceptual redesign of digital

convolution, with the goal of performing the bare minimum number of computations to calculate the filter output. This is done by decomposing the full filter into 'branches' of polyphase subfilters, each containing only a fraction of the total filter taps equal to the number of taps divided by the upsample factor. Rather than explicitly upsampling the input signal and convolving with the full filter, each output sample is computed by selecting the appropriate subfilter branch based on its position in the upsampled domain, and convolving only with that subfilter's taps. This reduces the number of multiplications per output sample by a factor equal to the upsample factor, representing a significant computational saving. State saving and phase offset tracking are maintained across blocks to ensure correct filter alignment at block boundaries. When running the benchmarking for Polyresampler with 32768 samples, we saw a drastic decrease in the runtime. Since we are now avoiding two major sources of wasted work through the upsampling and downsampling and computing output samples that are later discarded, this decrease is not surprising.

Bench_resample_reference/32768/101	1719.75
Bench_resample_reference/65536/101	3383.04
Bench_resample_reference/131072/101	6818.13
Bench_polyResampler/32768/101	0.01
Bench_polyResampler/65536/101	0.02
Bench_polyResampler/131072/101	0.04

Figure 1: PolyResampler Bench runtimes

² Since they produce both strong and weak Highs and Lows. For instance, a strong High may be followed by a weak Low that does not cross below 0, causing the simple threshold scheme to incorrectly interpret the pair as H-H rather than the correct H-L

OptimizedFilt: For further optimization after implementing the polyphase filter, we looked to the optimized filter from mono, as based on information from lab 4, filters were likely to carry the biggest overhead. To optimize this filter, we elected to implement loop unrolling and precomputed range. For pre-computed range, we divided the filter process between samples that need access to the previous state and samples that don't. These were then split into two different loops. The loop with states has more computations to account for state usage, as well as a precomputed boundary within it for the state portion of the samples. The loop without states is only doing convolution, and so is less computationally intensive. For loop unrolling, we do calculations within the loop in bursts of 4 samples. Based on lab 4 information, this provides a small but still tangible optimization as it reduces the number of loop control operations. By designing the optimizedFilter function in such a way, we observed a significant reduction in runtime from 11.42 seconds to 0.13 seconds for an input size of 32768. This improvement is because the optimized filter avoids computing many samples that would later be discarded by the decimation stage, as well as the addition of the two optimization techniques mentioned above. We can also see that as we scale the samples linearly, the runtime also scales linearly.

Bench_filtDecimate_reference/32768/101	11.42
Bench_filtDecimate_reference/65536/101	22.86
Bench_filtDecimate_reference/131072/101	45.78
Bench_optimizedFilt/32768/101	0.13
Bench_optimizedFilt/65536/101	0.26
Bench_optimizedFilt/131072/101	0.53

Figure 2: OptimizedFilt Bench Runtimes

Multithreading: With stereo working in real time, we then moved to set up multithreading to expedite RDS implementation. To do this, we grouped all RF functions into a separate file and thread named "rf_thread". We then did the same with stereo and mono, labelling it the "audio_thread". This didn't provide any optimization, as they still ran sequentially because audio requires RF data per block. However, we were able to optimize the audio pathway by running the mono and stereo paths in parallel using threading. This avoided any additional runtime caused by running the mono delay filter sequentially with the stereo path before mixing. To implement threading at this stage, we first implemented the audio and RF threads as functions imported from separate files. Once we verified that this worked, we swapped them in as threads. We ran into some minor compilation errors related to how we pass variables. To resolve this, we found we had to pass by reference using std::ref(variable) rather than &variable. Once this was fixed, threading prior to RDS was fully functional. Due to RDS's use of previously optimized filters and functions in the audio path, the optimizations primarily came from multithreading. All of RDS was made into a single thread to be run in the project file. This enabled it to be run in parallel with audio. This reduces total runtime by RDS's runtime, as neither had to wait for the other to finish before starting. Threading was also implemented in RDS the same way as in stereo. The parallel branches where RDS gets processed and subsequently combined with a delayed non-processed RDS signal have been threaded to run in parallel. This removes the delay block's runtime from the total runtime.

Analysis and Measurements

Mono MAC Operations

For all Mono modes, even though the filter length is increased by the expander when designing it, the polyphase implementation means we don't actually use all those coefficients every time. Instead, we

Mode	0	1	2	3
IQ Filter	240k* 101*2	288k* 101*2	240k* 101*2	384k* 101*2
Audio Filter	48k* 101	32k* 101	44.1k* 101	44.1k* 101
MAC/ Output Sample	1111	1919	1200	1860

Table 1: Mono MAC calculations

Stereo MAC Operations

$$\text{MAC/OutputSample} = \frac{\text{IFs} \times 2 \times 101 + \text{AudioFs} \times 101}{\text{Audio Fs}}$$

Handwritten notes: IFs x 2 x 101 (labeled I&Q Filter taps), AudioFs (labeled Filter taps)

Figure 3: Mono MAC Eq.

only use a subset of them for each output sample, so the number of MAC operations stays closer to the original number of taps rather than the full expanded length. The below chart summarizes the MAC calculations for Mono Modes 0-3, using the above equation.

For stereo MAC calculations, we will now use our previous value because we still must process mono, however we will add the additional filtering steps for Stereo, including the Pilot extraction, stereo Difference signal, and stereo filtering. We can ignore

$$\text{MAC/OS} = \text{Mono MAC} + \frac{\text{Pilot Ex.} \downarrow \text{Stereo Diff.} \downarrow \text{Stereo Filtering} \downarrow}{\text{Audio Fs}} \times (\text{IF Fs} \times 101 + \text{IF Fs} \times 101 + \text{Audio Fs} \times 101)$$

Figure 4: Stereo MAC eq.

the mixing as this adds 1 MAC per sample, as well as the combining of mono and stereo to extract L and R audio, as these add only 2 MAC operations per sample. We then can use the Above formula for calculating MAC operations across the 4 stereo modes.

RDS MAC Operations

For RDS modes, there is a significant increase in the number of MAC operations due to the increased number of filters. This includes the RF processing we saw in Mono and Stereo, with the addition of RDS Extraction filter, the Bandpass filter used at 114kHz after squaring the RDS band, the 3kHz LPF used after mixing the carrier and RDS band (which is performed on I and Q samples), and finally the rationale resampling done on both the I and Q samples. The MAC/OB (output bit) equation can be used.

Mode	0 (SPS = 16)	2 (SPS = 31)
RF processing	240k * 101 * 2	240k * 101 * 2
RDS Ex, BPF, LPF(I & Q)	240k * 101 * 4	240k * 101 * 4
Rational Resampling	2 * 101 * 16 * 2375	2 * 101 * 31 * 2375
MAC/OS	128,940	135,000

Figure 73: RDS MAC Calculations

Non-Linear Operations

For Mono, the only non-linear operation occurs in the RF front end processing, where we perform one atan2 operation for demodulation. This translates to (IF_Fs/Audio_Fs) atan2 operations per audio sample.

Mode	0	1	2	3
Atan2 operations	5	9	5.4	8.7

Figure 84: Mono non-linear operations

For fmPLL, the CPU cost depends on the sampling frequency, not the signal frequency, so we use the intermediate frequency of 240kHz for both Stereo and RDS PLL use cases. When analysing PLL, we perform 4 multiplications and additions, these are negligible compared to the non-linear operations we perform in the function. For every sample being processed, we perform an “atan2” operation, 2 “cos” operations, and 2 “sin” operations. For stereo, this translates to (IF_Fs/Audio_Fs) atan2 operations per audio sample, and (IF_Fs / Audio_Fs) x 2 cos and sin operations.

Mode	0	1	2	3
atan2 (PLL + FMdemod)	5 + 5 = 10	9 + 9 = 18	5.4 + 5.4 = 10.8	8.7 + 8.7 = 17.4
cos	10	18	10.8	17.4

Mode	0	1	2	3
Mono	1111	1919	1200	1860
Pilot	240k * 101	288k * 101	240k * 101	384k * 101
Stereo Diff.	240k * 101	288k * 101	240k * 101	384k * 101
Stereo Filt.	48k * 101	32k * 101	44.1k * 101	44.1k * 101
MAC/OS	2222	3838	2401	3720

Figure 5: Stereo MAC calculations

$$\text{MAC/OB} = \frac{\text{RF Processing} \downarrow \text{RDS Extraction BPF, LPF(I \& Q)} \downarrow \text{Rational Resampling (I \& Q)} \downarrow}{\text{IF Fs} \times 2 \times 101 + \text{IF Fs} \times 4 \times 101 + 2 \times 101 \times \text{SPS} \times 2375} \times \frac{\text{MAC/OS}}{\text{bits/sec}} = \text{MAC/bit}$$

Figure 6: RDS MAC eq.

Since the final output is measured In bits, which consists of 2 symbols, and one symbol consists of 16 or 31 samples, the base of the equation becomes small while the top remains large, causing us to see a drastic increase in the MAC operations per output sample for RDS.

sin	10	18	10.8	17.4
-----	----	----	------	------

Figure 95: Stereo non Linear Operations

For every RDS output bit, we perform $(IF_Fs / output_bits)$ atan2 operations, and $2 \times (IF_Fs / output\ Bits)$ cos and sin operations per output bit. As observed with the MAC operations for RDS, the number of non-linear operations is much higher than that of stereo. For modes 0 and 2, we have 404 of each the atan2, cos, and sin operations.

PI Measurements

Block	Mode 0 (ms)	Mode 1 (ms)	Mode 2 (ms)	Mode 3 (ms)
IQ Filter	1.5863 @600MHz 1.5709 @900MHz 1.1874 @1.2GHz 0.95502 @1.5GHz	2.05458 @600MHz 1.96492 @900MHz 1.7212 @1.2GHz 1.31752 @1.5GHz	1.70476 @600MHz 1.7051 @900MHz 1.31176 @1.2GHz 1.0745 @1.5GHz	3.00528 @600MHz 2.64256 @900MHz 2.2431 @1.2GHz 1.84998 @1.5GHz
Demodulation	0.78118 @600MHz 0.79494 @900MHz 0.5818 @1.2GHz 0.4734 @1.5GHz	1.72752 @600MHz 1.69202 @900MHz 1.25156 @1.2GHz 1.01826 @1.5GHz	0.89172 @600MHz 0.88528 @900MHz 0.66856 @1.2GHz 0.53568 @1.5GHz	1.5296 @600MHz 1.42984 @900MHz 0.84448 @1.2GHz 0.67938 @1.5GHz
Mono Output Filter	0.35764 @600MHz 0.32262 @900MHz 0.27658 @1.2GHz 0.22652 @1.5GHz	0.1662 @600MHz 0.1487 @900MHz 0.12836 @1.2GHz 0.1054 @1.5GHz	2.34124 @600MHz 2.26312 @900MHz 2.0091 @1.2GHz 1.81596 @1.5GHz	1.86368 @600MHz 1.724 @900MHz 1.55512 @1.2GHz 1.32854 @1.5GHz
Mono All Pass	0.0741 @600MHz 0.07256 @900MHz 0.06674 @1.2GHz 0.05752 @1.5GHz	0.04444 @600MHz 0.04206 @900MHz 0.04106 @1.2GHz 0.03418 @1.5GHz	0.12434 @600MHz 0.1156 @900MHz 0.10782 @1.2GHz 0.08562 @1.5GHz	0.04486 @600MHz 0.04158 @900MHz 0.03544 @1.2GHz 0.03356 @1.5GHz
Stereo Pilot Filter	1.45592 @600MHz 1.25122 @900MHz 1.07748 @1.2GHz 0.86512 @1.5GHz	1.73738 @600MHz 1.56186 @900MHz 1.30392 @1.2GHz 1.03374 @1.5GHz	1.38902 @600MHz 1.20124 @900MHz 1.04004 @1.2GHz 0.8251 @1.5GHz	2.31044 @600MHz 2.13668 @900MHz 1.91158 @1.2GHz 1.5999 @1.5GHz
Stereo Channel Filter	1.43722 @600MHz 1.22854 @900MHz 1.07606 @1.2GHz 0.86034 @1.5GHz	1.74438 @600MHz 1.53526 @900MHz 1.30788 @1.2GHz 1.03436 @1.5GHz	1.3928 @600MHz 1.16888 @900MHz 1.02008 @1.2GHz 0.82148 @1.5GHz	2.64842 @600MHz 2.2628 @900MHz 2.01394 @1.2GHz 1.56998 @1.5GHz
Stereo PLL	2.41566 @600MHz 1.90366 @900MHz 1.43496 @1.2GHz 1.14624 @1.5GHz	3.095 @600MHz 2.6039 @900MHz 2.3939 @1.2GHz 2.0146 @1.5GHz	2.66974 @600MHz 2.01978 @900MHz 1.50946 @1.2GHz 1.19838 @1.5GHz	2.9014 @600MHz 2.3977 @900MHz 1.82694 @1.2GHz 1.4505 @1.5GHz
Stereo Output Filter	0.39488 @600MHz 0.3478 @900MHz 0.27486 @1.2GHz 0.22382 @1.5GHz	0.1644 @600MHz 0.1414 @900MHz 0.1242 @1.2GHz 0.10202 @1.5GHz	2.20844 @600MHz 2.05248 @900MHz 1.52452 @1.2GHz 1.13534 @1.5GHz	2.0712 @600MHz 1.90596 @900MHz 1.38888 @1.2GHz 0.87092 @1.5GHz
RDS Resampler	2.18312 @600MHz 1.96426 @900MHz 1.70846 @1.2GHz 1.36402 @1.5GHz	N/A	9.20222 @600MHz 8.05336 @900MHz 6.93124 @1.2GHz 5.58258 @1.5GHz	N/A

Figure 10: [Raspberry Pi Runtime Measurements](#)

All runtime results were analyzed at 600 MHz since the other frequencies followed the same trends. For both the mono and stereo output filters, the runtimes in modes 2 and 3 are higher than expected based on the MAC calculations, which suggests there is extra overhead not captured in the theory, likely due to the polyphase resampler and the different upS/downS values. The demodulation and PLL runtimes match what we expected from the nonlinear analysis, with mode 0 being the fastest, followed by modes 2, 3, and 1. The mono all-pass filter runtime stays low across all modes since it has no MAC operations. The pilot and stereo channel filters also follow the expected MAC trends of 0, 2, 3 and 1, and their runtimes are very close across modes since they have the same structure of $IF_Fs \times N_taps$. Finally, the RDS resampler makes sense based on the MAC analysis, where mode 2 has a higher runtime due to the larger SPS value.

When analyzing the changes across frequencies, we can see that as the PI operating frequency increases our runtime decreases, which is expected since more clock cycles are available per second to process the same amount of data. Additionally, increasing the number of taps increases the runtime, since it directly increases the number of MAC operations, and decreasing the number of taps reduces the runtime.

Block	IQ Filter	Demod	Mono Output	Mono All Pass	Stereo Pilot	Stereo Channel	Stereo PLL	Stereo Output	RDS Resamp
13 Taps	0.1229	0.4734	0.0575	0.0575	0.1113	0.1107	1.1462	0.2238	0.1756
301 Taps	3.2785	0.67858	0.63212	0.0538	3.063	3.06842	1.43174	0.62666	4.8022

Figure 11: [Raspberry Pi Runtime Measurements, varied filter taps](#)

Decreasing the N_{taps} significantly reduces computational runtime but degrades overall signal quality by widening the transition band and reducing stopband attenuation. This poor isolation allows out-of-band noise to bleed through the system. Downstream, this spectral leakage causes background static in the mono audio and induces severe phase jitter in the stereo PLL, resulting in channel crosstalk. Most critically, because the RDS subcarrier operates at such low power, insufficient filtering causes massive inter-symbol interference that scrambles the constellation diagram, completely preventing reliable bit recovery and frame synchronization.

Improvement Proposals Further polyphase filter optimization: Our current implementation of the polyphase filter doesn't include all the potential optimization techniques studied in the course. We learned in our optimization of `optimizedFilter` that there is significant room for improvement in filter optimization. Techniques such as precomputed ranges and loop unrolling, as seen in `optimizedFilter`, could enable the program to run on even lower specifications. This could be implemented in a similar manner to `optimizedFilter` optimizations, as they follow a similar structure and logic. Further precomputation could be implemented by determining when you no longer need any information from the previous state block, which would lead to fewer operations within each for loop, saving time. This would then enable a simpler loop unrolling structure, as you could iterate k by 4 for both loops and convolve in units of 4 as opposed to 1 at a time. This would limit the number of loop operations, saving time.

PS name improvement: With the current implementation, the PS name doesn't exhibit the typical scrolling behaviour you would see in, for example, a car radio. This can make the display look a bit rough to read. To implement this, one idea could be to store the current PS name in an array. Once a new PS name has been entered with all segments filled, the new one could be pushed into the display, with the old one being pushed out. This would create the illusion of scrolling with clever timing, so that once the text has finished pushing the new PS name, a newer PS name is ready to be pushed. Because the RDS application wasn't found to be timing-intensive in our testing, it wouldn't cause the system to fail.

RDS Features: Additional RDS features, such as radio text, could be added to improve the usability of the program. In order to do this, we would need to check the group type and version of the RDS block that's being taken in. We would then read the corresponding blocks to output radio text. This would benefit the user by displaying more information, while expanding our own productivity by allowing us to further validate the functionality of RDS.

Project Activity

Week	Progress	Individual Contributions
1	Project was released	Studied Project Guidelines
2	Midterm recess	None
3	Mono Mode 0 Working	Logan – Mono mode 0 implementation Justin – Mono first iteration ³ , home R-Pi environment set-up Matthew – Mono model James – mono model

³ The details of each iteration are clearly labelled and discussed in the Implementations section of the report

4	Began work on stereo model, mono debugging for modes 1-3	Logan – Stereo Model (PLL state saving, needed to get a better understanding of the function of PLL and try state saving various parts of PLL.), Mono mode debugging Justin – Mono second iteration (blocks were not processing correct data, needed to implement phase_Offset across blocks.) Matthew – Mono model debugging James – Mono test cases (single pass only)
5	Stereo model working, mono working for all modes on computer	Logan – Stereo model (needed to debug stereo, analyse graphs without knowledge of how they should look, in order to deduce where the problem was and solve it.), mono mode debugging Justin – Mono third iteration, Stereo unthreaded implementation (Learning polyphase resampling technique and programming it.) Matthew – Stereo model and PLL James – Benchmarking polyresampler & optimizedFilter functions vs. slow method. Testing across blocks.
6	Mono working on Pi, finished Stereo implementation, Stereo working on Pi, threading implemented, RDS model	Logan – Stereo threading (Needed to go through documentation of std::threads to understand compilation issues from pass-by-reference into threads), optimization to run on Pi Justin – RDS modelling iterations 1, 2, and 3. Matthew – Stereo thread (rf and audio threads) benchmarking and testing James – debugging RDS model (compared bitstream after CDR and Manchester decode, found skipped symbols in RDS pipeline, caused by ignoring first symbol of block at times.)
7	Finished RDS implementation on Pi in all modes and settings, Finished benchmarking and testing	Logan – RDS application layer (Importing RDS application model into integration, understanding differences between how Python handles classes compared to C++), RDS threading Justin – RDS model final iteration, RDS computer and real-time implementation debugging (designed test cases to ensure rds_helper functions resolved previous issues for state saving.) Matthew – RDS helpers, thread implementation, Testing /benchmarking James – RDS implementation + threading, testing for RDS helpers
8	Wrote the report, did in-class presentation, did in-lab examinations	Logan – Introduction, Conclusion, Project Overview, Implementation Details for all previous individual contributions, Improvement Proposals Justin – Implementation section for Mono, Stereo implementation, RDS modelling and implementation. PolyResampler section. Matthew – Measurement & Analysis, runtime measuring on Pi James – Analysis, MAC operations, implementation for RDS / threading

FIGURE 12: PROJECT ACTIVITY TABLE

Conclusion

The 3DY4 radio receiver project was a challenging but highly rewarding project. We experienced all sorts of challenges throughout, regarding both personal and technical hurdles. Even facing unique challenges exclusive to our group, such as difficult group specific constraints, we were able to work together to overcome and finish the project, with a functional RDS and Stereo output on Raspberry Pi 4 with minimum specs. There are improvements that could be made to our design. Some functions aren't optimized to the extent that they could be, and the user experience with RDS could be further improved. However, we learned a lot about both technical aspects of this project, such as audio processing, alongside lessons we will be able to apply in all aspects of our lives and careers, such as project organization and time management.

References

1 Professor Roberts, "Timing Recovery & Frame Sync," WRAP DSP Lecture 3, Wireless Research and Applied Programming (WRAP), Apr. 1, 2026. [Online]. Available: https://drive.google.com/open?id=1Ambl38MOcvMq6_cIzaddivvAlmoVeEq